Bilan du projet Gastronome Créatif

Jeudi 8 octobre 2026 - ASP.NET Core MVC, Entity Framework Core et PostgreSQL

État d'avancement : la récupération des ingrédients et des quantités est codée. Leur affichage dans la vue Razor et le test final restent à effectuer.

1. Objectif de la séance

Poursuivre la page de détail d'une recette. Les informations générales étaient déjà affichées : nom, photo, pseudonyme du créateur, temps de préparation, temps de cuisson et difficulté. Notre objectif était d'ajouter les ingrédients et leurs quantités.

2. Relations entre les tables

Table	Colonnes et rôle
ingredients	id_ingredient : identifiant ; nom : nom de l'ingrédient
ingredients_recettes	id_ingredient : ingrédient ; id_recette : recette ; quantite : quantité nécessaire

La table ingredients_recettes représente une relation plusieurs-à-plusieurs : une recette possède plusieurs ingrédients et un ingrédient peut être utilisé dans plusieurs recettes. La quantité est spécifique à chaque association (par exemple 200 g, 15 cl ou 1 pincée).

3. Modèle Models/Ingredients.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace GC_MVC.Models
{
    [Table("ingredients")]
    public class Ingredients
    {
        [Key]
        public int id_ingredient { get; set; }
        public string nom { get; set; } = string.Empty;
    }
}
```

Notions : [Table] associe la classe à la table PostgreSQL ; [Key] indique la clé primaire ; string.Empty initialise une chaîne non nullable.

4. Modèle Models/Ingredients_recettes.cs

```
using System.ComponentModel.DataAnnotations.Schema;

namespace GC_MVC.Models
{
    [Table("ingredients_recettes")]
    public class Ingredients_recettes
    {
        public int id_ingredient { get; set; }
        public int id_recette { get; set; }
        public string quantite { get; set; } = string.Empty;
    }
}
```

Nous avons corrigé une faute de frappe dans le nom de la classe. Sa clé primaire est composée de id_ingredient et id_recette ; elle sera configurée dans GCContext.

5. Modification de Data/GCContext.cs

```
public DbSet<Ingredients> Ingredients { get; set; }
public DbSet<Ingredients_recettes> Ingredients_Recettes { get; set; }
```

Ces propriétés donnent accès aux tables via _context.Ingredients et _context.Ingredients_Recettes.

```
modelBuilder.Entity<Ingredients_recettes>().HasKey(cr => new
{
    cr.id_recette,
    cr.id_ingredient
});
```

La clé composée est déclarée dans OnModelCreating, sur le même principe que Categories_recettes. La structure de la base PostgreSQL n'a pas été modifiée.

6. Nouveau ViewModel IngredientQuantiteViewModel.cs

```
namespace GC_MVC.ViewModels
{
    public class IngredientQuantiteViewModel
    {
        public string Nom { get; set; } = string.Empty;
        public string Quantite { get; set; } = string.Empty;
    }
}
```

Chaque objet regroupe le nom d'un ingrédient et la quantité correspondante, même si ces informations proviennent de deux tables distinctes.

7. Requête LINQ écrite dans RecettesController.cs

```
List<IngredientQuantiteViewModel> ingredients = (
    from ingredient in _context.Ingredients
    join association in _context.Ingredients_Recettes
        on ingredient.id_ingredient equals association.id_ingredient
    where association.id_recette == id
    select new IngredientQuantiteViewModel
    {
        Nom = ingredient.nom,
        Quantite = association.quantite
    }
).ToList();
```

Élément LINQ	Fonction
from	Point de départ : les ingrédients
join ... equals	Jointure des deux tables sur id_ingredient
where	Filtre sur la recette consultée (id)
select new	Création d'objets IngredientQuantiteViewModel
ToList()	Exécution de la requête et matérialisation des résultats dans une liste

Une List est adaptée parce qu'une recette peut avoir plusieurs ingrédients, un seul ou aucun.

8. Enrichissement de RecetteDetailViewModel.cs

```
using GC_MVC.Models;

namespace GC_MVC.ViewModels
{
    public class RecetteDetailViewModel
    {
        public Recettes? Recette { get; set; }
        public Utilisateurs? Createur { get; set; }
        public List<IngredientQuantiteViewModel> Ingredients { get; set; }
            = new List<IngredientQuantiteViewModel>();
    }
}
```

Le ViewModel principal regroupe désormais les informations générales de la recette, son créateur et la liste des ingrédients avec leurs quantités. Il prépare les données nécessaires à la vue, sans ajouter de colonnes artificielles à la table recettes.

9. Assemblage final dans l'action Detail(int id)

L'action recherche la recette, retourne NotFound() si elle n'existe pas, recherche le créateur, récupère les ingrédients par LINQ, crée le ViewModel puis retourne la vue.

```
RecetteDetailViewModel vm = new RecetteDetailViewModel
{
    Recette = recette,
    Createur = createur,
    Ingredients = ingredients
};

return View(vm);
```

Ingredients (majuscule) est une propriété du ViewModel ; ingredients (minuscule) est la variable locale contenant les résultats de la requête. Le using Npgsql inutilisé peut être supprimé du contrôleur.

10. Point de reprise pour demain

Nous nous sommes arrêtés juste avant l'exercice Razor dans Views/Recettes/Detail.cshtml. Il faudra :

• Ouvrir la vue Detail.cshtml.

• Ajouter un titre « Ingrédients ».

• Parcourir Model.Ingredients avec une boucle foreach.

• Afficher dans chaque <li> la quantité et le nom de l'ingrédient.

• Lancer l'application et vérifier le résultat dans le navigateur.

À retenir : modèles C#, annotations de mapping, clé primaire composée, DbSet, jointure LINQ, filtrage, projection, ToList(), listes typées, ViewModels et transmission des données du contrôleur vers Razor.